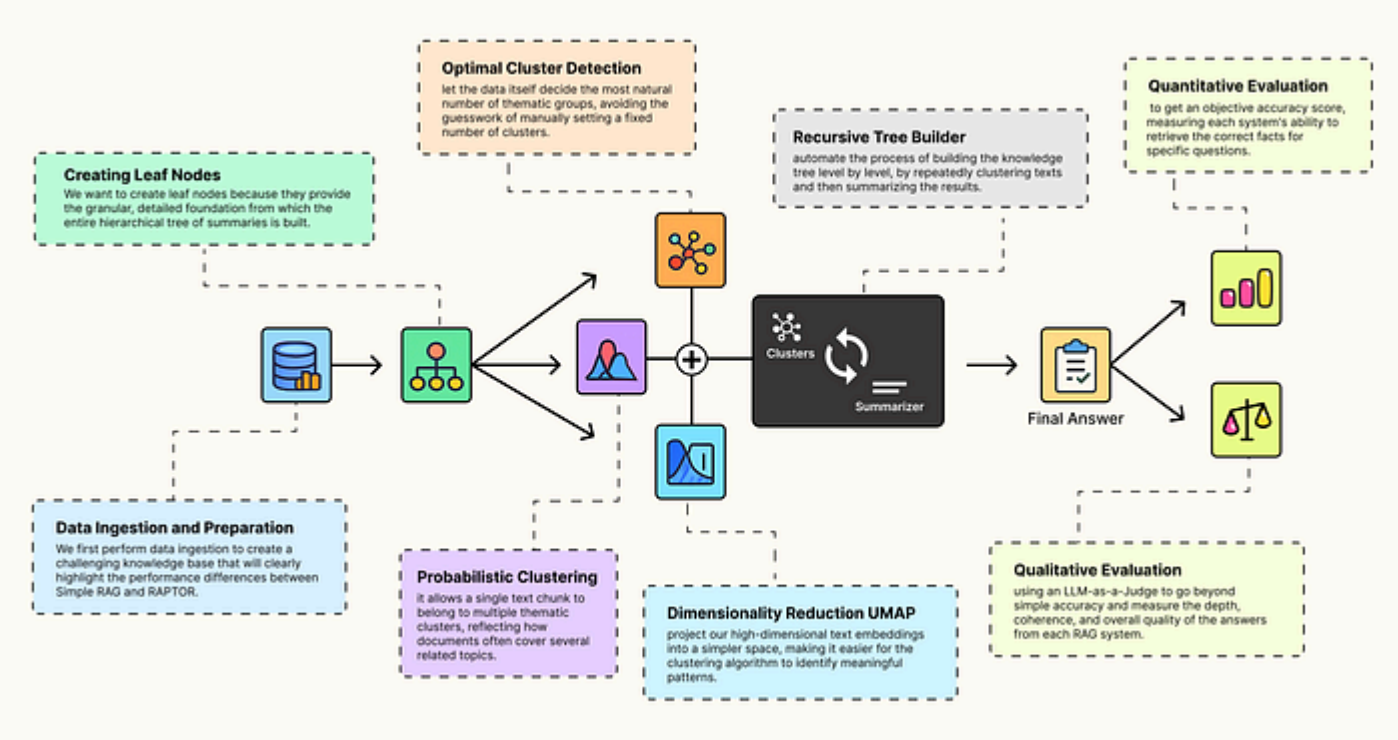


Optimizing RAG with the Smarter Indexing RAPTOR Pipeline

Probabilistic Clustering, UMAP, Tree Structure and more

Fareed Khan

Level Up Coding · ~32 min read · September 3, 2025 (Updated: September 3, 2025) ·
Free: No

Read this story for free: link

There are tons of RAG optimization techniques you can use to improve performance, from query transformations to re-ranking models. The challenge is that each new layer often brings extra complexity, more LLM calls, and more moving parts to your architecture.

> But what if we could get a better performance by focusing on just one thing, building a smarter index?

This is the core idea behind **RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval)**, it keeps RAG simple at query time while delivering superior results by building a hierarchical index that mirrors human understanding from details to high-level concepts.

Here is a high-level overview of how RAPTOR works:

1. **Start with Leaf Nodes:** First, we break down all source documents into small, detailed chunks. These are the foundational **"leaf nodes"** of our knowledge

2. **Cluster for Themes:** Then, we use an Machine Learning clustering algorithms (Like GMM/Agglomerative etc.) to automatically group these leaf nodes into related clusters based on their semantic meaning.
3. **Summarize for Abstraction:** We use an LLM to generate a concise, high-quality summary for each cluster. These summaries become the next, more abstract layer of the tree.
4. **Recurse to Build Upwards:** We repeat the clustering and summarization process on the newly created summaries, building the tree level by level towards higher concepts.
5. **Index Everything Together:** Finally, we combine all text, the original leaf chunks and all generated summaries into a single **"collapsed tree"** vector store for a powerful, multi-resolution search.

In this blog, we are going to ...

> Evaluate a simple RAG pipeline against a RAPTOR-based RAG pipeline and explore why RAPTOR performs better than other approaches.

All the code is available in my GitHub repository.

Implementation of RAPTOR based RAG...

A Step-by-Step Implementation of RAPTOR based RAG implementation - FareedKhan-dev/rag-with-raptor

github.com

The codebase is organized as follows:

```
Copy

rag-with-raptor/
├── LICENSE
├── README.md
├── requirements.txt
├── rag_vs_raptor.ipynb  # Comparison Eval
└── raptor_guide.ipynb   # Raptor standalone implementation
```

Table of Contents

Initializing our RAG Configuration

The two most important components of any RAG system are:

1. An embedding model → to convert documents into vector space for retrieval.
2. A text generation model (LLM) → to interpret retrieved content and produce answers.

ago.

If we used a **newer LLM**, it might already "know" the answers internally, bypassing retrieval. By choosing an older model, we ensure that the evaluation truly tests **retrieval quality** which is exactly where RAPTOR vs simple RAG makes a difference.

We first need to import PyTorch and other supporting components:

Copy

```
# Import the core PyTorch library for tensor operations
import torch

# Import LangChain's wrappers for Hugging Face models
from langchain_huggingface import HuggingFaceEmbeddings, HuggingFacePipeline

# Import core components from the transformers library for model loading and co
from transformers import AutoModelForCausalLM, AutoTokenizer, pipeline, BitsAnd

# Import LangChain's tools for prompt engineering and output handling
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
```

representations.

```
Copy

# --- Configure Embedding Model ---
embedding_model_name = "sentence-transformers/all-MiniLM-L6-v2"

# Use GPU if available, otherwise fallback to CPU
model_kwargs = {"device": "cuda"}

# Initialize embeddings with LangChain's wrapper
embeddings = HuggingFaceEmbeddings(
    model_name=embedding_model_name,
    model_kwargs=model_kwargs
)
```

This embedding model is small but perfect for large-scale document indexing without excessive memory usage. Next for generation, we are using Mistral-7B-Instruct v0.2, a capable but compact instruction-tuned model.

To make it memory-friendly, we load it with **4-bit quantization** using `BitsAndBytesConfig`.

```
Copy
```

```
# Quantization: reduces memory footprint while preserving performance
quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type="nf4"
)
```

We now need to load the tokenizer and the LLM itself with the quantization settings applied.

Copy

```
# Load tokenizer
tokenizer = AutoTokenizer.from_pretrained(llm_id)

# Load LLM with quantization
model = AutoModelForCausalLM.from_pretrained(
    llm_id,
    torch_dtype=torch.float16,
    device_map="auto",
    quantization_config=quantization_config
)
```

Hugging Face **pipeline** for text generation.

```
Copy

# Create a text-generation pipeline using the loaded model and tokenizer.
pipe = pipeline(
    "text-generation",
    model=model,
    tokenizer=tokenizer,
    max_new_tokens=512 # Controls the max length of the generated summaries and
)
```

Finally, we wrap the Hugging Face pipeline in LangChain's `HuggingFacePipeline` so it integrates smoothly with our retrieval pipeline later.

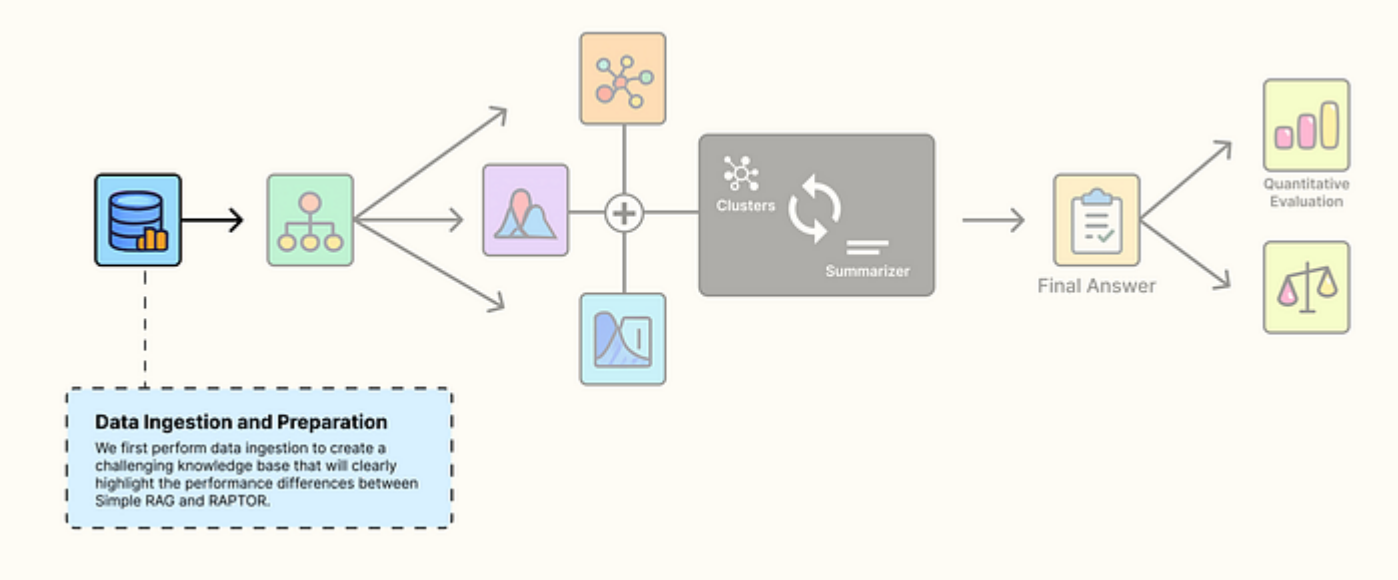
```
Copy

# Wrap pipeline for LangChain compatibility
llm = HuggingFacePipeline(pipeline=pipe)
```

Great! Now that we have configured the basic LLM and embedding model, which are the two main components of a RAG pipeline, we can move on to defining our complex knowledge base for RAG.

Data Ingestion and Preparation

To properly showcase how **RAPTOR** can improve **RAG** performance, we need a **complex and challenging database.** The idea is that when we run queries against it, we want to see real differences between **simple RAG** and **RAPTOR-enhanced RAG.**



Data Ingestion Step (Created by Fareed Khan)

For this reason, we are focusing on the **Hugging Face documentation.** The docs are rich in overlapping information and contain subtle variations that can easily trip up a naïve retriever.

- `trainer.save_model()`
- `unwrap_model().save_pretrained()`
- `zero_to_fp32()`

All of these refer to the same underlying concept, consolidating model shards into a full checkpoint.

> A simple RAG pipeline might retrieve only one of these variants and **miss the broader context**, leading to incomplete or even broken instructions. RAPTOR, on the other hand, can consolidate and reason across them.

Since Hugging Face has an extensive documentation ecosystem, we are narrowing down to five **core guides** where most practical usage happens.

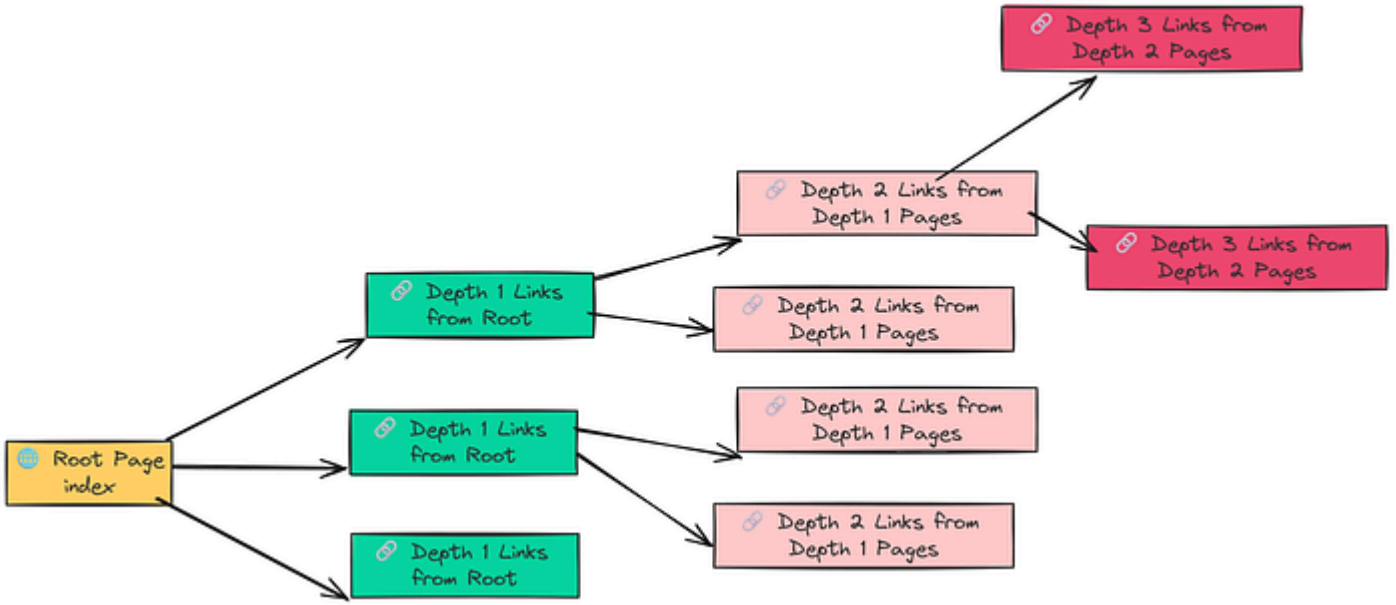
Let's initialize their URLs.

Copy

```
# Define the documentation sections to scrape, with varying crawl depths.
urls_to_load = [
    {"url": "https://huggingface.co/docs/transformers/index", "max_depth": 3},
```

```
    {"url": "https://huggingface.co/docs/perf/index", "max_depth": 1},
    {"url": "https://huggingface.co/docs/accelerate/index", "max_depth": 1}
]
```

We are targeting five well-known areas of Hugging Face usage. A key parameter here is `max_depth`, which controls how deeply we crawl from the starting page, let's visually understand how this parameter works.



Depth parameter work (Created by Fareed Khan)

- It starts with the root page (... `docs/transformers/index`).

- Then, it crawls into the links found inside those subpages → this is depth 2.
- Finally, it continues one more level into the links within those sub-subpages → this is depth 3.

Now that we have defined the URLs, the next step is to actually fetch the content. For this, we will use LangChain `RecursiveUrlLoader` in combination with BeautifulSoup to cleanly extract the text from each page.

Copy

```
from langchain_community.document_loaders import RecursiveUrlLoader
from bs4 import BeautifulSoup as Soup

# Empty list to append components
docs = []

# Iterate through the list and crawl each documentation section.
for item in urls_to_load:
    # Initialize the loader with the specific URL and parameters.
    loader = RecursiveUrlLoader(
        url=item["url"],
        max_depth=item["max_depth"],
        extractor=lambda x: Soup(x, "html.parser").text, # Use BeautifulSoup to
        prevent_outside=True, # Ensure we stay within the documentation pages
        use_async=True, # Use asynchronous requests for faster crawling
        timeout=600, # Set a generous timeout for slow pages
```

```
    loaded_docs = loader.load()
    docs.extend(loaded_docs)
    print(f"Loaded {len(loaded_docs)} documents from {item['url']}")
```

We are basically looping through all the docs and grabbing their content and most parameters are fairly straightforward, but two deserve special attention:

- `prevent_outside=True` → Not to scrape irrelevant assets like external JavaScript, analytics, or unrelated links. We want only the core documentation text.
- `timeout=600` → Hugging Face pages can be heavy or slow at times. A generous timeout prevents premature failures and allows us to crawl granularly without the risk of being blocked or cut off.

When we run this loop we get the following output.

Copy

```
###### OUTPUT #######
Loaded 68 documents from https://huggingface.co/docs/transformers/index
Loaded 35 documents from https://huggingface.co/docs/datasets/index
Loaded 21 documents from https://huggingface.co/docs/tokenizers/index
Loaded 12 documents from https://huggingface.co/docs/peft/index
Loaded 9 documents from https://huggingface.co/docs/accelerate/index
```

So, we have basically total of `145` documents which are enough for us to evaluate our raptor based rag. Let's do some further analysis on our scraped data like counting the tokens.

Copy

```
import numpy as np

# We need a consistent way to count tokens, using the LLM's tokenizer is the mo
def count_tokens(text: str) -> int:
    """Counts the number of tokens in a text using the configured tokenizer."""
    # Ensure text is not None and is a string
    if not isinstance(text, str):
        return 0
    return len(tokenizer.encode(text))

# Extract the text content from the loaded LangChain Document objects
docs_texts = [d.page_content for d in docs]

# Calculate token counts for each document
token_counts = [count_tokens(text) for text in docs_texts]

# Print statistics to understand the document size distribution
print(f"Total documents: {len(docs_texts)}")
print(f"Total tokens in corpus: {np.sum(token_counts)}")
print(f"Average tokens per document: {np.mean(token_counts):.2f}")
```

This is what a basic tokens analysis on our data is.

```
Copy

######### OUTPUT #########
Total documents: 145
Total tokens in corpus: 312566
Average tokens per document: 2155.59
Min tokens in a document: 312
Max tokens in a document: 12453
```

The maximum tokens per document is 12K, which is quite large and may lead to loss of information if we don't apply chunking. With a total of around 300K tokens, the size is substantial, so we need to determine an optimal chunking strategy for our knowledge base. A simple way to do this is by plotting the token distribution to identify the optimal chunk size.

```
Copy

# Set the size of the plot for better readability.
plt.figure(figsize=(10, 6))

# Create the histogram.
```

```
# - color='blue': Sets the color of the bars.
# - alpha=0.7: Sets the transparency of the bars.
plt.hist(token_counts, bins=50, color='blue', alpha=0.7)

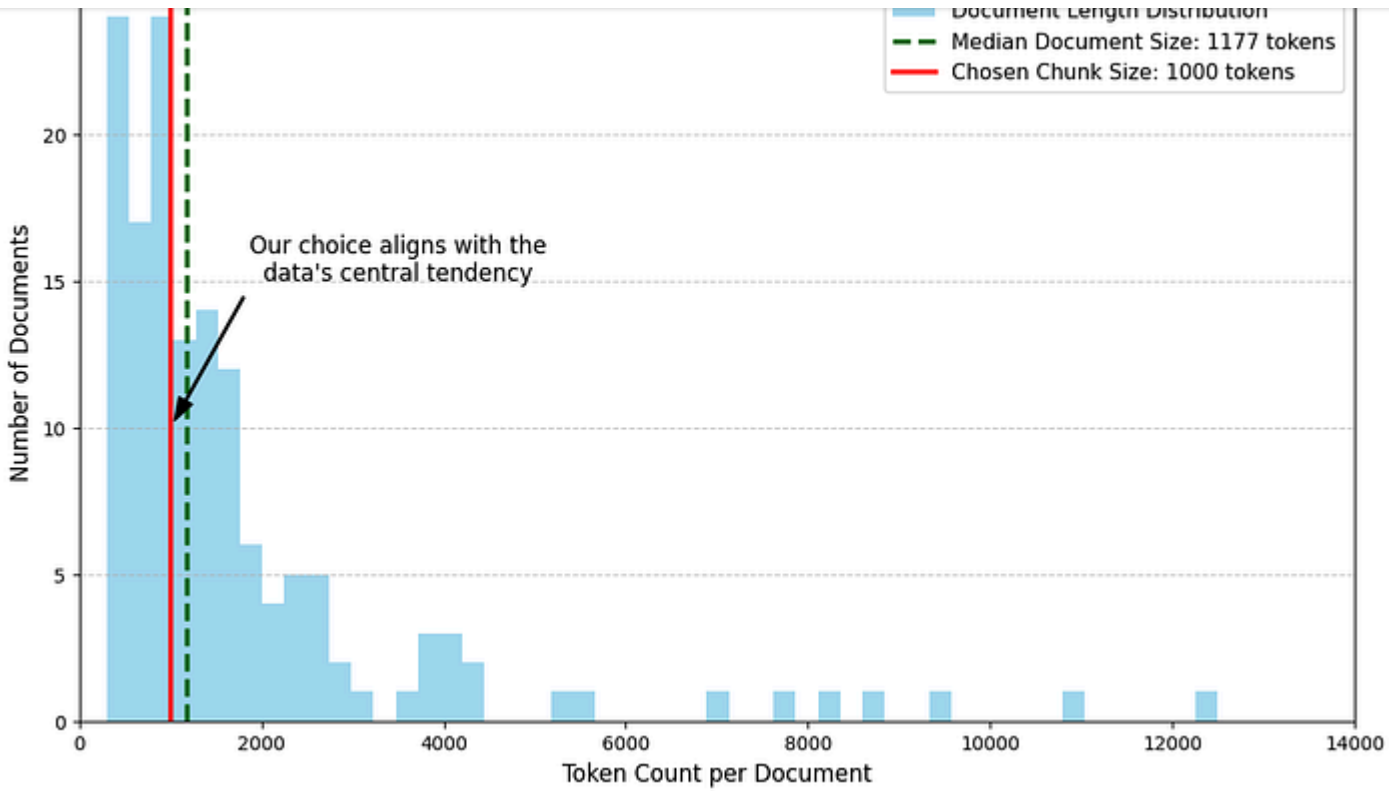
# Set the title of the histogram.
plt.title('Distribution of Document Token Counts')

# Label the x-axis.
plt.xlabel('Token Count')

# Label the y-axis.
plt.ylabel('Number of Documents')

# Add a grid to the background for easier reading of values.
plt.grid(True)

# Display the generated plot.
plt.show()
```



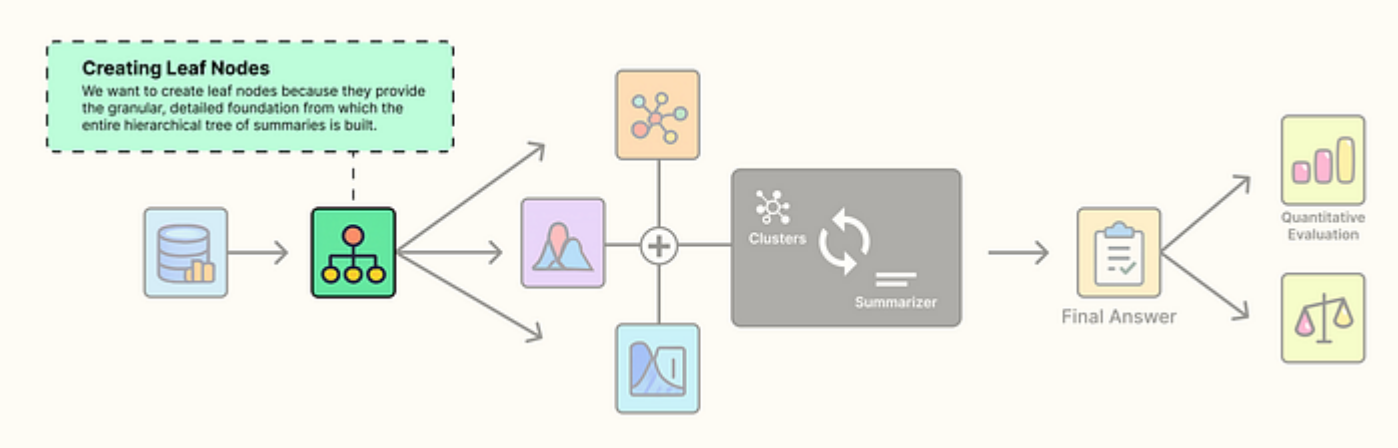
Token Distribution (Created by Fareed Khan)

From the plot, we can see that the optimal value roughly the median of our document sizes is around 1000 tokens. Therefore, we will use **1000** as our chunk size. In the next section, we will perform document chunking, which also serves as the first component of the Raptor Tree architecture.

Creating Leaf Nodes of RAPTOR Tree

Our analysis of the document token counts made one thing clear, many of our 145 documents are far too large to be used directly in a RAG system. The histogram pointed us toward an optimal chunk size of around 1000 tokens.

> This initial chunking step is the first and most critical part of the RAPTOR process.

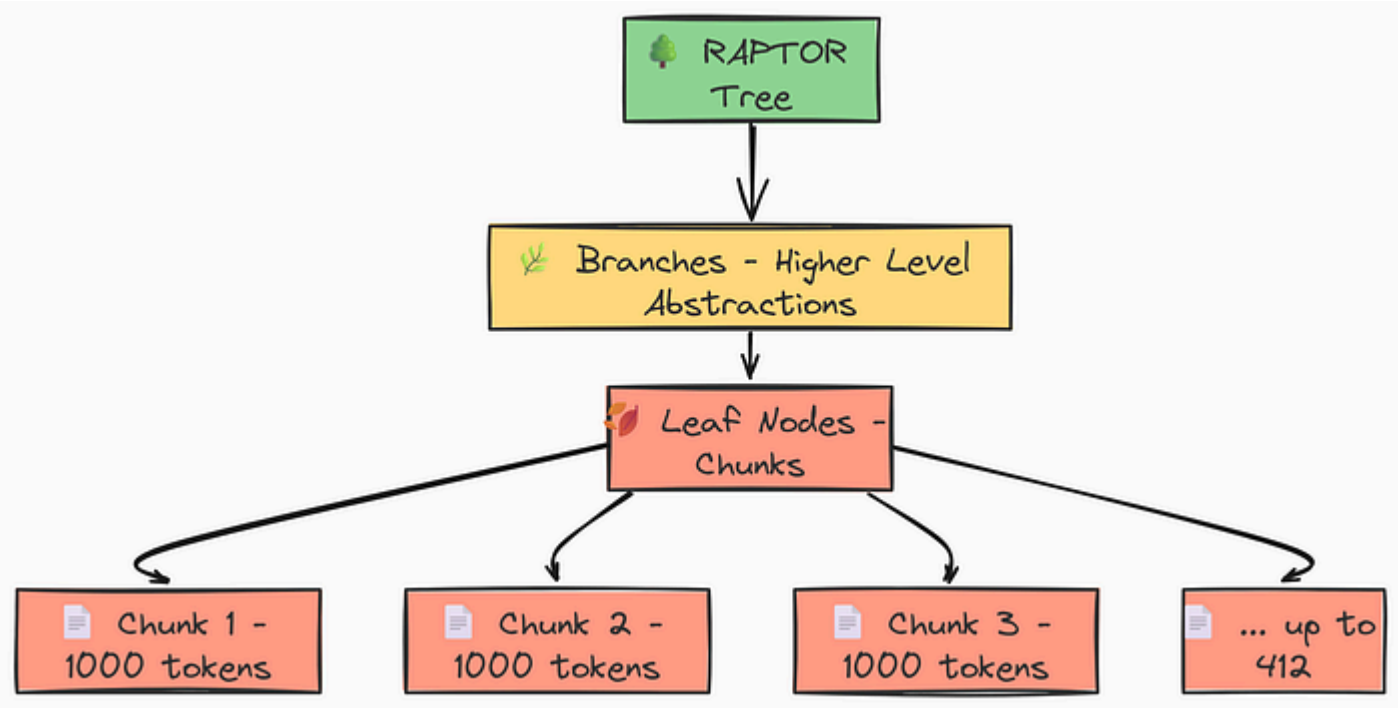


Leaf Nodes Step (Created by Fareed Khan)

What is the Point of Leaf Nodes?

these are our **leaf nodes.**

They are the granular, Level 0 chunks that contain the raw details from the source documents: specific code examples, API function descriptions, and precise definitions.



Leaf Nodes of RAPTOR Tree (Created by Fareed Khan)

and trunk of the tree. But without high-quality leaves, the entire structure would be weak.

To create them, we'll use LangChain `RecursiveCharacterTextSplitter` configured with our LLM's tokenizer. This ensures our splits are intelligent and respect token boundaries.

Copy

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

# We join all the documents into a single string for more efficient processing.
# The '---' separator helps maintain document boundaries if needed later.
concatenated_content = "\n\n --- \n\n".join(docs_texts)

# Create the text splitter using our LLM's tokenizer for accuracy.
text_splitter = RecursiveCharacterTextSplitter.from_huggingface_tokenizer(
    tokenizer=tokenizer,
    chunk_size=1000, # The max number of tokens in a chunk
    chunk_overlap=100  # The number of tokens to overlap between chunks
)

# Split the text into chunks, which will be our leaf nodes.
leaf_texts = text_splitter.split_text(concatenated_content)
```

Let's break down the key parameters we used here:

- `from_huggingface_tokenizer(tokenizer=tokenizer)` : This makes the splitter "token-aware," so `chunk_size=1000` refers to tokens, not characters, which is far more accurate for our models.
- `chunk_size=1000` : This directly implements the decision we made from our histogram analysis, creating chunks of a manageable size.
- `chunk_overlap=100` : This is a crucial technique to prevent losing context at the boundaries where chunks are split. Each chunk shares 100 tokens with the previous one.

Running this code gives us our foundational layer.

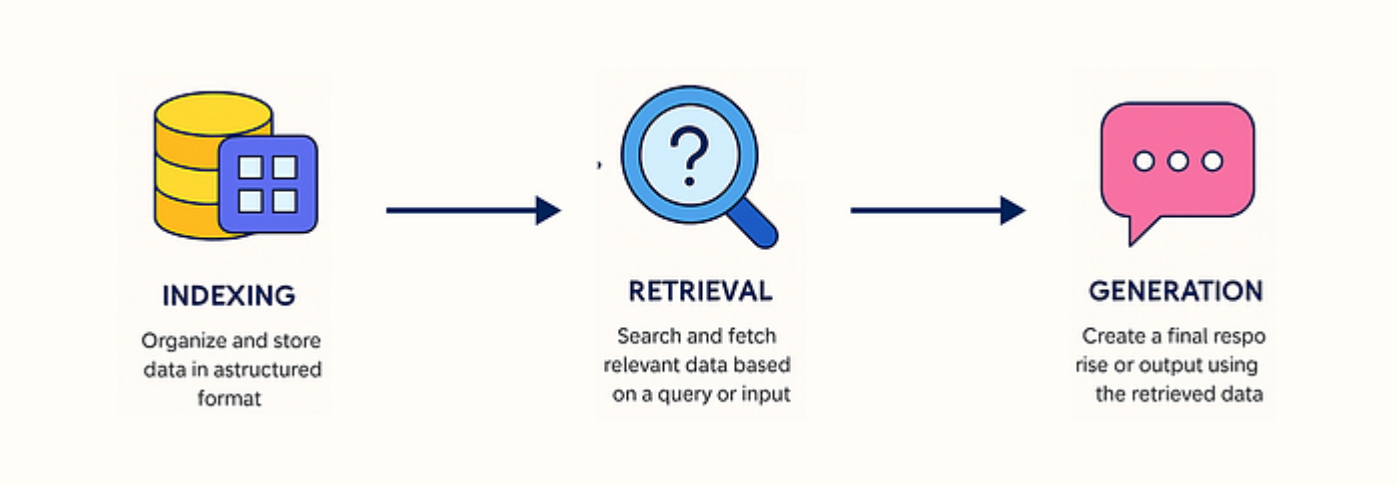
Copy

```
#### OUTPUT #####
Created 412 leaf nodes (chunks) for the RAPTOR tree.
```

We have transformed our 145 large documents into 412 focused, granular leaf nodes. Before we build the rest of the RAPTOR tree.

Implementing Simple RAG and Its Problems

To prove that RAPTOR is truly an improvement, we need something to compare it against. We will now build a standard, non-hierarchical RAG system. This **"Simple RAG"** will use the exact same models and the exact same 412 leaf nodes we just created. Its knowledge base will contain only these leaves, ensuring a fair comparison.



Simple RAG (Created by Fareed Khan)

Copy

```
from langchain_community.vectorstores import FAISS

# In a simple RAG, the vector store is built only on the leaf-level chunks.
vectorstore_normal = FAISS.from_texts(
    texts=leaf_texts,
    embedding=embeddings
)

# Create a retriever from this vector store that fetches the top 5 results.
retriever_normal = vectorstore_normal.as_retriever(
    search_kwargs={'k': 5}
)

print(f"Built Simple RAG vector store with {len(leaf_texts)} documents.")


### OUTPUT ###
Built Simple RAG vector store with 412 documents.
```

Now that we have the retriever, let's build a full RAG chain and see how it performs on a high-level question. This will immediately highlight the limitations of this simple approach.

```
from langchain_core.runnables import RunnablePassthrough

# This prompt template instructs the LLM to answer based ONLY on the provided c
final_prompt_text = """You are an expert assistant for the Hugging Face ecosyst
Answer the user's question based ONLY on the following context. If the context
CONTEXT:
{context}
QUESTION:
{question}
ANSWER:"""
final_prompt = ChatPromptTemplate.from_template(final_prompt_text)

# A helper function to format the retrieved documents.
def format_docs(docs):
    return "\n\n".join(doc.page_content for doc in docs)

# Construct the RAG chain for the simple approach.
rag_chain_normal = (
    {"context": retriever_normal | format_docs, "question": RunnablePassthrough
    | final_prompt
    | llm
    | StrOutputParser()
)

# Let's ask a broad, conceptual question.
question = "What is the core philosophy of the Hugging Face ecosystem?"
answer = rag_chain_normal.invoke(question)
```

Here, we pipe the retriever's output into our prompt and then to the LLM. Let's examine the result.

```
Copy

#### OUTPUT ###

Question: What is the core philosophy of the Hugging Face ecosystem?

Answer: The Hugging Face ecosystem is built around the `transformers`
library, which provides APIs to easily download and use pretrained models.
The core idea is to make these models accessible. For example, the `pipeline`
function is a key part of this, offering a simple way to use models for
inference. It also includes libraries like `datasets` for data loading and
`accelerate` for training.
```

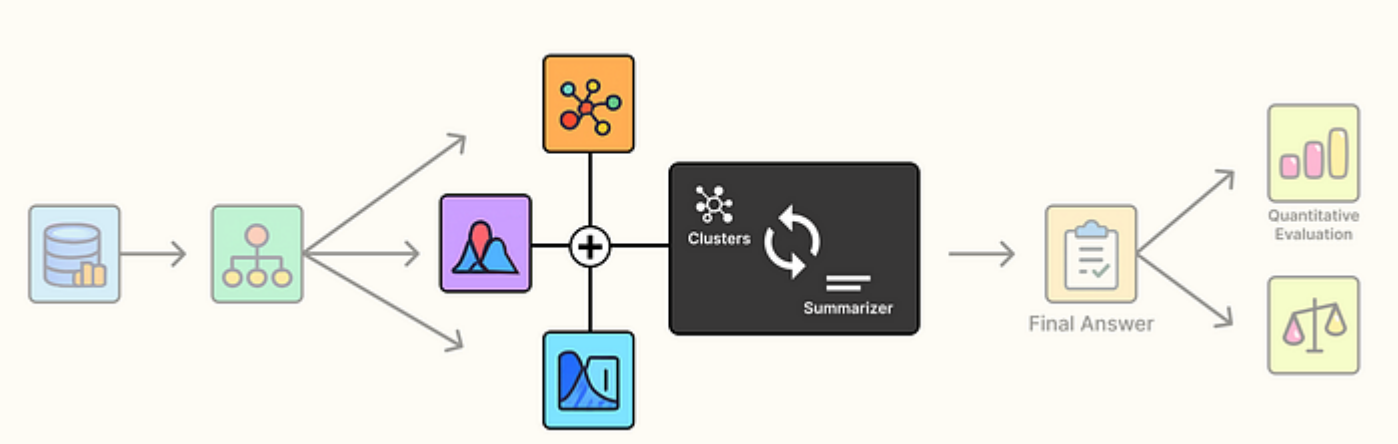
> This answer isn't wrong, but it's disjointed. It feels like a collection of random facts stitched together.

It talks about `pipeline`, `datasets`, and `accelerate`, but doesn't explain how they work together or the bigger goals of accessibility, interoperability, and efficiency. This is a "lost in the details" problem—the retriever pulled in small pieces with the right words but missed the main idea.

sophisticated clustering engine.

Building a Hierarchical Clustering Engine

We have seen that simple retrieval isn't enough. To build the RAPTOR tree, we need a way to group our 412 leaf nodes into meaningful, thematic clusters. For example, all chunks discussing `Trainer` arguments should be grouped together, and all chunks about `PEFT` and `LoRA` should form another group.



Heirarchical Process (Created by Fareed Khan)

The RAPTOR paper proposes a sophisticated, multi-stage clustering process that is far more robust than a simple K-Means approach. It involves three key

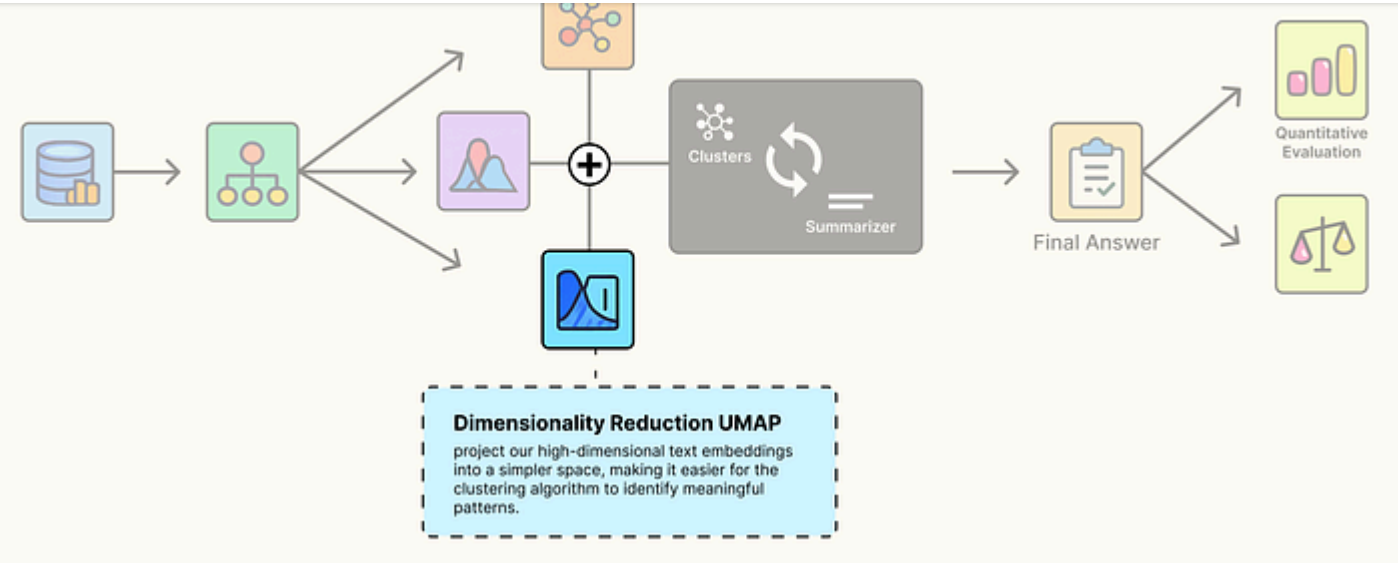
1. **Dimensionality Reduction (UMAP):** To help the clustering algorithm see the "shape" of the data more clearly.
2. **Optimal Cluster Detection (GMM + BIC):** To let the data decide how many clusters it naturally has.
3. **Probabilistic Clustering (GMM):** To assign chunks to clusters based on probabilities, allowing a single chunk to belong to multiple related topics.

Let's build this engine entirely.

Dimensionality Reduction with UMAP

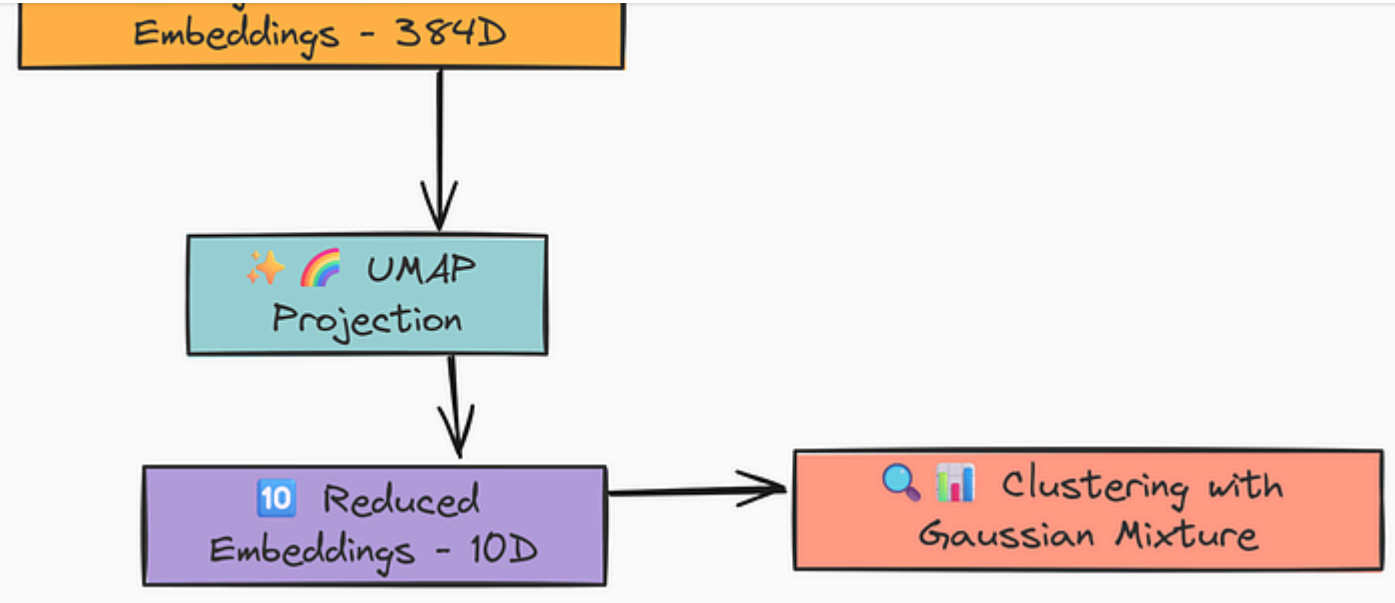
Our text embeddings exist in a high-dimensional space (384 dimensions for our model).

> This can make it hard for clustering algorithms to work well, a problem often called the "Curse of Dimensionality"



Dimensionality Reduction (Created by Fareed Khan)

We use **UMAP (Uniform Manifold Approximation and Projection)** to project our embeddings into a lower-dimensional space (e.g., 10 dimensions) while preserving the semantic relationships between them.



UMAP Approach (Created by Fareed Khan)

This gives the clustering algorithm a clearer, less noisy "map" of the data to work with.

Copy

```
from typing import Dict, List, Optional, Tuple
import numpy as np
import pandas as pd
import umap
from sklearn.mixture import GaussianMixture

def global_cluster_embeddings(embeddings: np.ndarray, dim: int, n_neighbors: Op
```

```
    if n_neighbors is None:
        n_neighbors = int((len(embeddings) - 1) ** 0.5)
    # Return the UMAP-transformed embeddings
    return umap.UMAP(
        n_neighbors=n_neighbors,
        n_components=dim,
        metric=metric,
        random_state=RANDOM_SEED
    ).fit_transform(embeddings)
```

We are basically taking our high-dimensional embeddings and reduces them to a specified `dim`. We use the `cosine` metric because it's highly effective for measuring similarity in text-based vector spaces.

Let's quickly look into the parameters and what they are doing here:

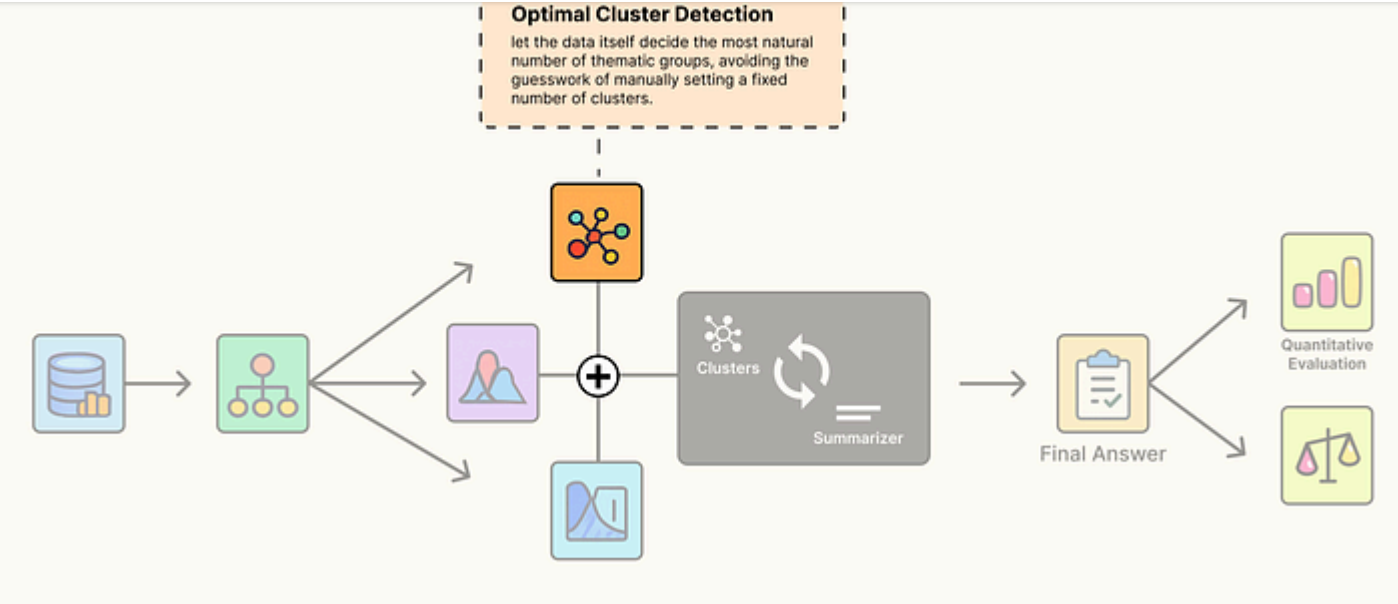
- `n_neighbors`: This parameter controls how UMAP balances local versus global structure. A smaller value focuses on finer details, while a larger value looks at the broader structure. The heuristic we use is a common starting point.
- `n_components=dim`: This specifies our target—the number of dimensions we want to reduce our data to.

- `random_state=RANDOM_SEED` : This ensures that the UMAP algorithm produces the exact same projection every time we run the code, which is critical for reproducible results.
- The `.fit_transform(embeddings)` method then performs the reduction. It first `fit` s the UMAP model by learning the underlying structure of the high-dimensional data, then `transform` s it into the new, lower-dimensional space.

Optimal Cluster Number Detection

Now comes another question, How many clusters should we create? 5? 10? 50? Picking a fixed number (`k`) is arbitrary and rarely optimal.

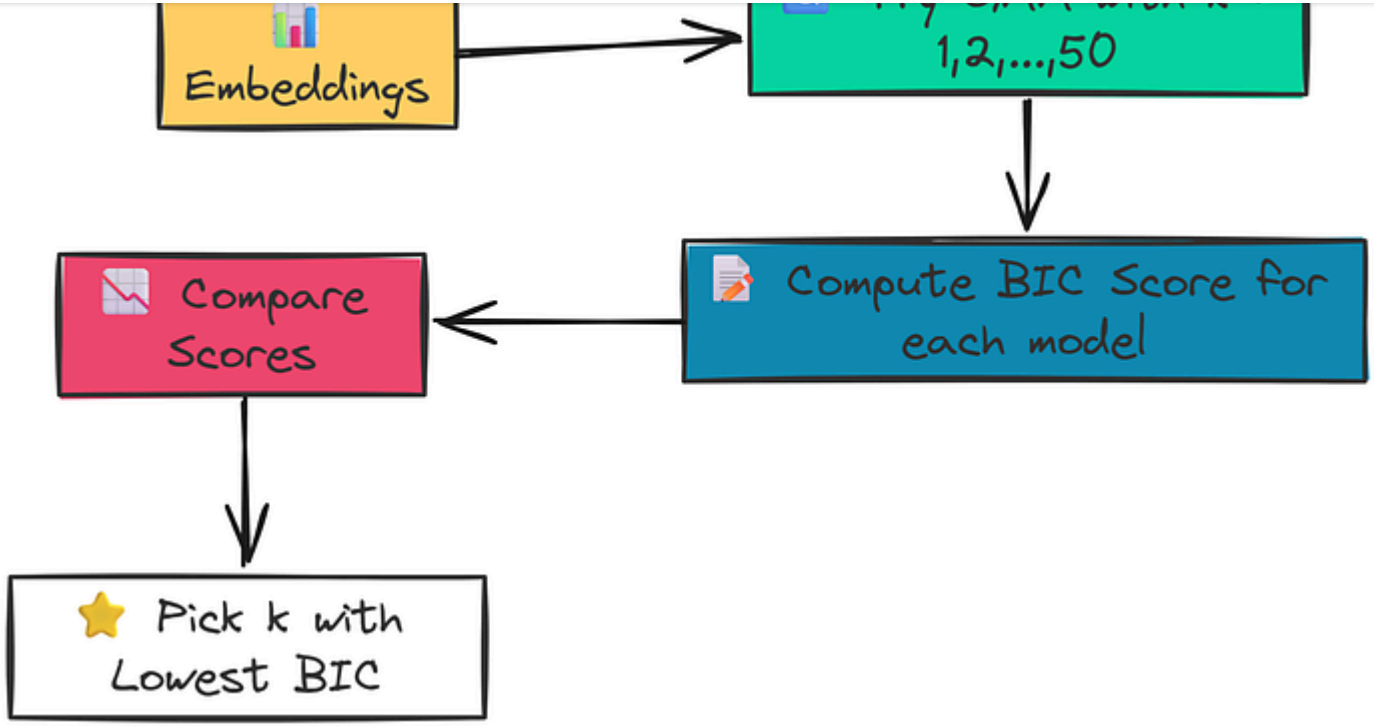
> A better approach is to let the data tell us its natural number of groupings.



Natural Cluster Detection (Created by Fareed Khan)

We achieve this using a **Gaussian Mixture Model (GMM)** and the **Bayesian Information Criterion (BIC)**.

The process involves fitting several GMMs with a different number of clusters and using the BIC score to find the best balance between model fit and model complexity.



Cluster Number Optimal (Created by Fareed Khan)

The lowest BIC score indicates the optimal number of clusters.

Copy

```
def get_optimal_clusters(embeddings: np.ndarray, max_clusters: int = 50) -> int
    """Determine the optimal number of clusters using the Bayesian Information
    # Limit the max number of clusters to be less than the number of data point
    max_clusters = min(max_clusters, len(embeddings))
    # If there's only one point, there can only be one cluster
```

```
    # Test different numbers of clusters from 1 to max_clusters
    n_clusters_range = np.arange(1, max_clusters)
    bics = []
    for n in n_clusters_range:
        # Initialize and fit the GMM for the current number of clusters
        gmm = GaussianMixture(n_components=n, random_state=RANDOM_SEED)
        gmm.fit(embeddings)
        # Calculate and store the BIC for the current model
        bics.append(gmm.bic(embeddings))

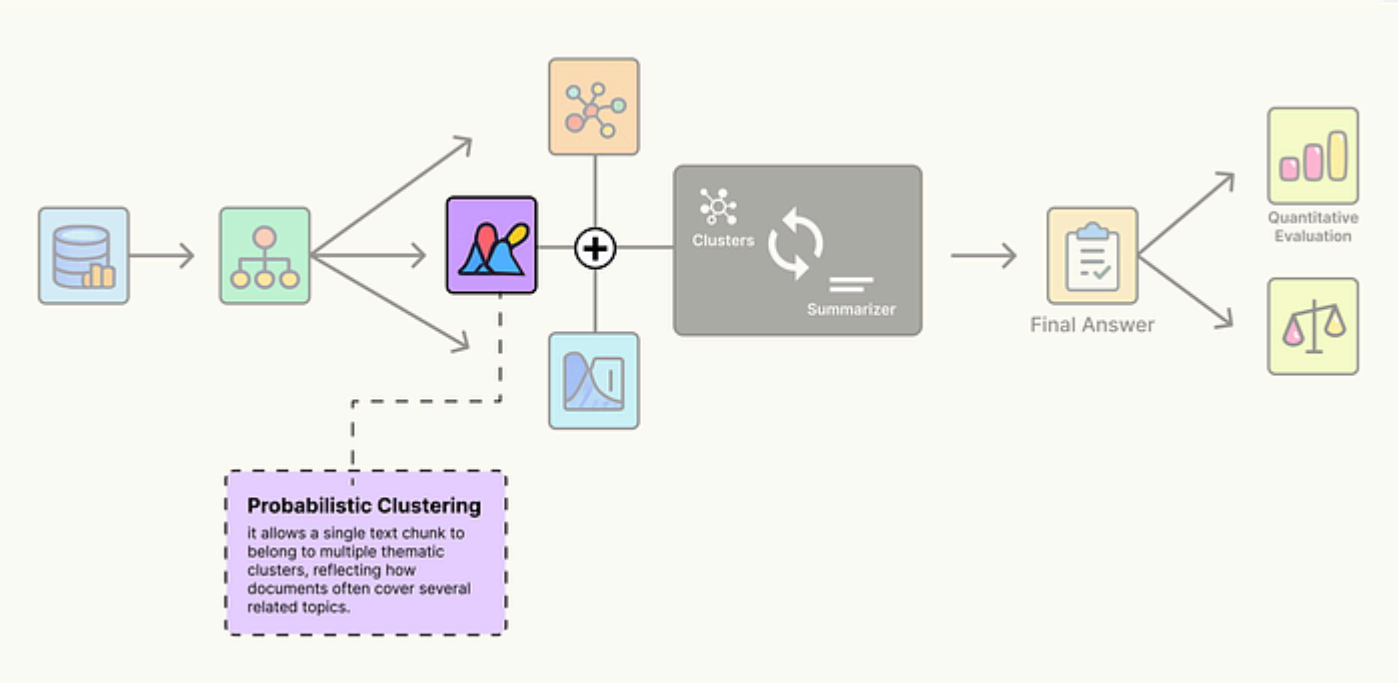
    # Return the number of clusters that resulted in the lowest BIC score
    return n_clusters_range[np.argmin(bics)]
```

The logic inside this function is a systematic search for the best model:

1. It iterates through a range of possible cluster counts (e.g., from 1 to 50).
2. In each iteration, it initializes and trains a `GaussianMixture` model with that specific number of clusters (`n_components=n`).
3. After training, it calculates the `gmm.bic(embeddings)` for that model. The BIC score rewards a good fit but penalizes complexity (having too many clusters).
4. Finally, after testing all options, `np.argmin(bics)` finds the index of the lowest BIC score. We use this index to retrieve the corresponding cluster count, which is our optimal `k` .

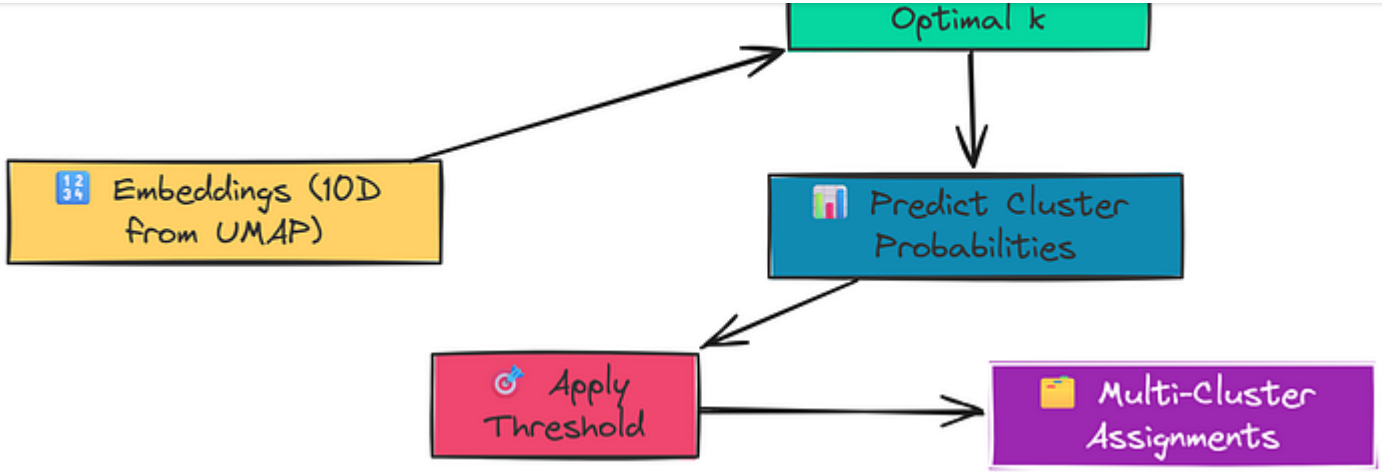
Probabilistic Clustering with GMM

Now we put it all together. Unlike algorithms like K-Means which perform **"hard clustering"** (assigning a data point to exactly one cluster), a GMM performs **"soft clustering"**. It calculates the probability that a point belongs to each cluster.



Probabilistic Clustering (Created by Fareed Khan)

This is perfect for our use case. A single text chunk might discuss both model training and data pre-processing, so it should belong to both clusters.



Probablistic Clustering (Created by Fareed Khan)

We can achieve this by setting a probability threshold.

Copy

```
def GMM_cluster(embeddings: np.ndarray, threshold: float) -> Tuple[List[np.ndar
    """Cluster embeddings using a GMM and a probability threshold."""
    # Find the optimal number of clusters for this set of embeddings
    n_clusters = get_optimal_clusters(embeddings)

    # Fit the GMM with the optimal number of clusters
    gmm = GaussianMixture(n_components=n_clusters, random_state=RANDOM_SEED)
    gmm.fit(embeddings)

    # Get the probability of each point belonging to each cluster
    probs = gmm.predict_proba(embeddings)
```

```
    # A single point can be assigned to multiple clusters.
    labels = [np.where(prob > threshold)[0] for prob in probs]

    return labels, n_clusters
```

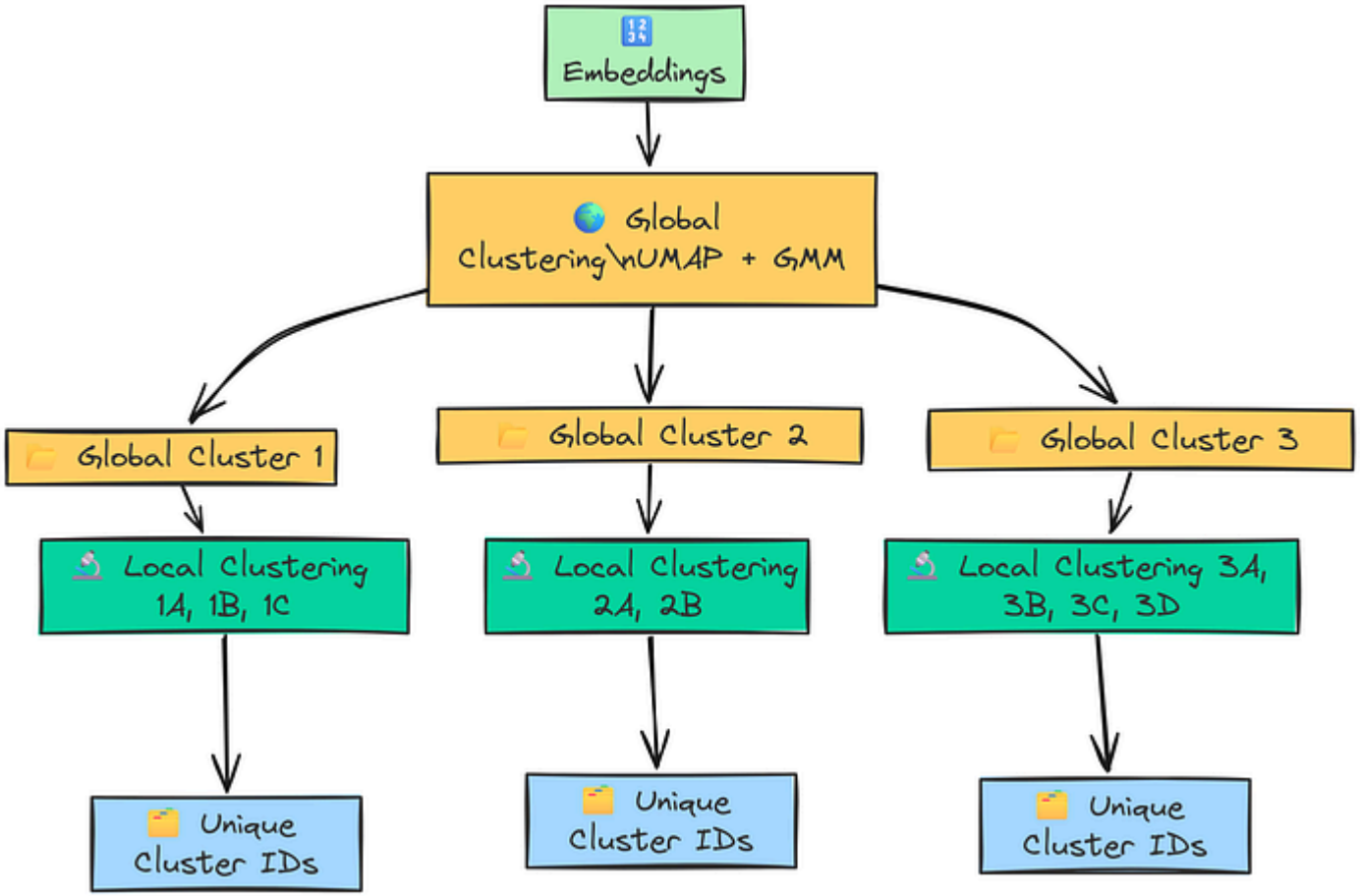
This function ties everything together:

1. First, it calls `get_optimal_clusters` to determine the best number of clusters for the input embeddings.
2. It then fits a `GaussianMixture` model with this optimal number.
3. The critical step is `gmm.predict_proba(embeddings)`. This returns a matrix where each row represents a text chunk and each column contains the probability that the chunk belongs to that cluster.
4. The final line, `[np.where(prob > threshold)[0] for prob in probs]`, processes this probability matrix. For each chunk's list of probabilities (`prob`), `np.where` finds the indices of the clusters that exceed our `threshold`. This gives us a list of cluster IDs for each chunk, achieving our multi-label assignment.

With these three components, UMAP, BIC, and GMM we have our clustering engine ready. Now, we need to orchestrate them to build the hierarchical structure of our tree.

Hierarchical Clustering Orchestrator

Having our building blocks for clustering is great, but the real power comes from how we arrange them. The RAPTOR paper proposes a clever two-stage process:



Hierarchical Clustering (Created by Fareed Khan)

"Datasets Library," and "Training and Optimization."

2. **Local Clustering:** Then, we zoom in. For each of these global clusters, we run the clustering process again on just the documents within that cluster. This finds more specific sub-topics. The **"Training and Optimization"** cluster might break down into smaller local clusters for **PEFT**, **Accelerate** and **Trainer Arguments**.

This global-then-local approach allows us to capture both the forest and the trees. We'll now write a single orchestrator function, `perform_clustering`, that implements this entire hierarchical logic.

Copy

```
def perform_clustering(embeddings: np.ndarray, dim: int = 10, threshold: float
    """Perform hierarchical clustering (global and local) on the embeddings."""
    # Handle cases with very few documents to avoid errors during dimensionalit
    if len(embeddings) <= dim + 1:
        return [np.array([0]) for _ in range(len(embeddings))]

    # --- Global Clustering Stage ---
    # First, reduce the dimensionality of all embeddings globally.
    reduced_embeddings_global = global_cluster_embeddings(embeddings, dim)
    # Then, perform GMM clustering on the reduced-dimensional data.
    global_clusters, n_global_clusters = GMM_cluster(reduced_embeddings_global,
```

```
# Initialize a list to hold all final local cluster assignments for each d
all_local_clusters = [np.array([]) for _ in range(len(embeddings))]
# Keep track of the total number of clusters found so far to ensure unique
total_clusters = 0

# Iterate through each global cluster to find sub-clusters.
for i in range(n_global_clusters):
    # Get all original indices for embeddings that are part of the current
    global_cluster_indices = [idx for idx, gc in enumerate(global_clusters)
    if not global_cluster_indices:
        continue

    # Get the actual embeddings for this global cluster.
    global_cluster_embeddings_ = embeddings[global_cluster_indices]

    # Perform local clustering on this subset of embeddings.
    if len(global_cluster_embeddings_) <= dim + 1:
        local_clusters, n_local_clusters = ([np.array([0])] * len(global_cl
    else:
        # We don't need a separate 'local_cluster_embeddings' function.
        # The global one works, as it adapts n_neighbors to the input size.
        reduced_embeddings_local = global_cluster_embeddings(global_cluster
        local_clusters, n_local_clusters = GMM_cluster(reduced_embeddings_l

    # Map the local cluster results back to the original document indices.
    for j in range(n_local_clusters):
        # Find which documents within the local set belong to this specific
        local_cluster_indices = [idx for idx, lc in enumerate(local_cluster
        if not local_cluster_indices:
```

```
            # Get the original indices from the full dataset.
            original_indices = [global_cluster_indices[idx] for idx in local_cl
            # Assign the new, globally unique cluster ID to these documents.
            for idx in original_indices:
                all_local_clusters[idx] = np.append(all_local_clusters[idx], j

        # Increment the total cluster count to ensure the next global cluster's
        total_clusters += n_local_clusters

    return all_local_clusters
```

This function is the main part of of our clustering logic. Let's break down its flow:

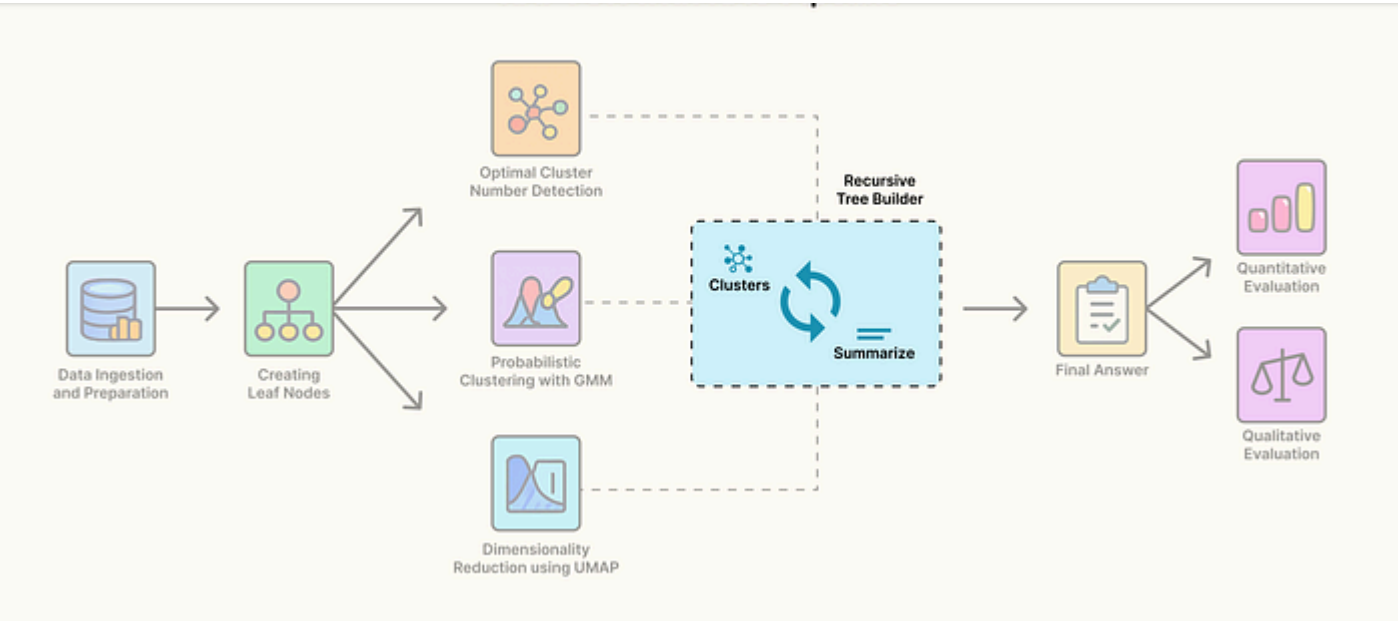
1. **Global Phase:** It starts by running `global_cluster_embeddings` and `GMM_cluster` on the entire set of input embeddings to find `n_global_clusters`.
2. **Local Loop:** It then iterates from `i = 0` to `n_global_clusters - 1`.
3. **Subset Selection:** Inside the loop, for each global cluster `i`, it gathers the indices and the actual embeddings of all documents that belong to it.
4. **Re-cluster:** It runs the *entire clustering process again* (UMAP -> GMM) on this smaller subset of embeddings, discovering local sub-clusters.
5. **Label Mapping:** This is the most complex part. It carefully maps the local cluster IDs (e.g., 0, 1, 2) to a globally unique ID. It does this by adding the `total_clusters` offset. So, the local clusters from the first global cluster might

6. **Return Value:** The function returns `all_local_clusters`, a list where each element corresponds to an original document and contains an array of its final, unique cluster IDs.

So now we can now assemble the final recursive function that will build the RAPTOR tree, level by level.

Building and Executing the RAPTOR Tree

We now have all the necessary pieces: leaf nodes (Level 0) and a powerful hierarchical clustering engine. The final step is to combine them in a process that builds the RAPTOR tree from the bottom up.



RAPTOR Tree (Created by Fareed Khan)

This process is driven by two key actions: **abstraction** (summarization) and **recursion**.

The Abstraction Engine: Summarization

Before we build the recursive function, we need to define the "A" in RAPTOR: the **Abstractive** component. This is a mechanism that takes a cluster of related text chunks and uses an LLM to synthesize them into a single, high-quality summary.

tree, representing the distilled essence of all its child documents. This is how we move up the ladder of abstraction from specific details to broader concepts.

We'll create a simple LangChain Expression Language (LCEL) chain for this. The prompt is engineered to instruct the LLM to act as an expert technical writer, ensuring our summaries are coherent, detailed, and capture the main ideas.

Copy

```
from langchain.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# Define the summarization chain
summarization_prompt = ChatPromptTemplate.from_template(
    """You are an expert technical writer.
    Given the following collection of text chunks from the Hugging Face documer
    Focus on the main concepts, APIs, and workflows described.
    CONTEXT: {context}
    DETAILED SUMMARY:"""
)

# Create the summarization chain by piping the prompt to the LLM and then to ar
summarization_chain = summarization_prompt | llm | StrOutputParser()
```

The Recursive Tree Builder

The recursive function, `recursive_build_tree`, will orchestrate the entire process. At each level, it will:

1. **Cluster** the input texts.
2. **Summarize** each cluster using our abstraction engine.
3. **Recurse** by using the newly generated summaries as the input for the next level.

This continues until we can no longer form meaningful clusters or we reach a predefined maximum depth.

Copy

```
def recursive_build_tree(texts: List[str], level: int = 1, n_levels: int = 3) -
    """The main recursive function to build the RAPTOR tree using all component
    results = {}
    # Base case: stop if max level is reached or no texts to process
    if level > n_levels or len(texts) <= 1:
        return results

    # --- Step 1: Embed and Cluster the texts for the current level ---
```

```
df_clusters = pd.DataFrame({'text': texts, 'cluster': cluster_labels})

# --- Step 2: Prepare for Summarization ---
# Since a text can belong to multiple clusters, we need to 'explode' the Da
expanded_list = []
for _, row in df_clusters.iterrows():
    for cluster_id in row['cluster']:
        expanded_list.append({'text': row['text'], 'cluster': int(cluster_

if not expanded_list: # Stop if no clusters were formed
    return results

expanded_df = pd.DataFrame(expanded_list)
all_clusters = expanded_df['cluster'].unique()
print(f"--- Level {level}: Generated {len(all_clusters)} clusters ---")

# --- Step 3: Summarize each cluster using our pre-defined chain ---
summaries = []
for i in all_clusters:
    # Get all texts for the current cluster
    cluster_texts = expanded_df[expanded_df['cluster'] == i]['text'].tolist
    # Join the texts into a single context string
    formatted_txt = "\n\n---\n\n".join(cluster_texts)
    # Generate a summary for the cluster
    summary = summarization_chain.invoke({"context": formatted_txt})
    summaries.append(summary)

# Store the summaries in a DataFrame for this level
df_summary = pd.DataFrame({'summaries': summaries, 'cluster': all_clusters]
```

```
    # --- Step 4: Recurse ---
    # Check if we should continue to the next level
    if level < n_levels and len(all_clusters) > 1:
        new_texts = df_summary["summaries"].tolist()
        # Call the function again on the summaries, incrementing the level
        next_level_results = recursive_build_tree(new_texts, level + 1, n_leve
        results.update(next_level_results)

    return results
```

This function is the master conductor of the entire RAPTOR process. Let's trace its execution:

1. **Base Case:** It first checks if it should stop. If the current `level` exceeds our maximum desired depth (`n_levels`) or if there's only one piece of text left to process, the recursion ends.
2. **Cluster:** It takes the input `texts` , embeds them, and uses our `perform_clustering` orchestrator to assign cluster labels.
3. **Summarize:** It then iterates through each unique cluster ID. For each cluster, it gathers all the text chunks belonging to it and passes them to our `summarization_chain` to produce a new, abstract summary.
4. **Store Results:** It saves the cluster assignments and the newly generated summaries for the current level.

the output of one level as the input for the next.

Now, let's execute this function on our initial 412 leaf nodes and watch the tree grow.

```
Copy

# Execute the RAPTOR process on our chunked leaf_texts.
# This will build a tree with a maximum of 3 levels of summarization.
raptor_results = recursive_build_tree(leaf_texts, level=1, n_levels=3)


#### OUTPUT ####
--- Level 1: Generated 8 clusters ---
Level 1, Cluster 0: Generated summary of length 2011 chars.
Level 1, Cluster 1: Generated summary of length 1954 chars.
... (and so on for all 8 clusters) ...
--- Level 2: Generated 3 clusters ---
Level 2, Cluster 0: Generated summary of length 2050 chars.
... (and so on for all 3 clusters) ...
```

The output shows the process in action.

- **Level 1:** The function took our 412 leaf nodes and grouped them into 8 thematic clusters. It then generated 8 new summaries, one for each.

and generated 3 corresponding top-level summaries.

- The process stopped because the next level would have too few documents (3) to form meaningful clusters, satisfying our base case.

We have successfully built our hierarchical data structure. The final step is to index all of this knowledge like the leaves, the branches, and the trunk into a single vector store that our RAG system can query.

Indexing with the Collapsed Tree Strategy

We have built a multi-level tree of knowledge, but how do we make it searchable? Instead of creating a complex graph database, RAPTOR uses an elegant and highly effective technique called the **"collapsed tree."**

The idea is simple, we take all the text from every level of the tree the original 412 leaf nodes, the 8 summaries from Level 1, and the 3 summaries from Level 2 and put them all into a single, unified list.

When we perform a similarity search, our query is simultaneously compared against the detailed leaf nodes, the mid-level summaries, and the high-level

> This multi-resolution index lets the retrieval system find the perfect level of abstraction for any given question.

Let's implement this. We'll gather all our texts and build the final RAPTOR vector store.

Copy

```
from langchain_community.vectorstores import FAISS

# Start with a copy of the original leaf texts.
all_texts_raptor = leaf_texts.copy()

# Add the summaries from each level of the RAPTOR tree.
for level in raptor_results:
    # Get the summaries from the current level's results
    summaries = raptor_results[level][1]['summaries'].tolist()
    # Add them to our master list
    all_texts_raptor.extend(summaries)

# Build the final vector store using FAISS.
vectorstore_raptor = FAISS.from_texts(
    texts=all_texts_raptor,
    embedding=embeddings
)

# Create the final retriever for the RAPTOR RAG system.
```

```
print(f"Built RAPTOR vector store with {len(all_texts_raptor)} total documents

#### OUTPUT ####
Built RAPTOR vector store with 423 total documents (leaves + summaries).
```

Our RAPTOR knowledge base is now indexed and ready. It contains the original 412 leaf nodes plus the 11 summaries (8 from level 1 + 3 from level 2), for a total of 423 documents.

Now let's build a RAG chain for our RAPTOR system and test it with questions at different levels of abstraction. We expect it to outperform our simple RAG baseline, especially on conceptual questions.

Copy

```
# Create the RAG chain for the RAPTOR approach.
rag_chain_raptor = (
    {"context": retriever_raptor | format_docs, "question": RunnablePassthrough
    | final_prompt
    | llm
    | StrOutputParser()
)
```

We will now ask three types of questions to demonstrate the system's capabilities.

This is a detailed question that should retrieve a specific **leaf node** containing a code example.

Copy

```
question_specific = "How do I use the `pipeline` function in the Transformers l
answer = rag_chain_raptor.invoke(question_specific)
print(answer)
#### OUTPUT ####
The `pipeline` function is the easiest way to use a pre-trained model for a giv
Here is a simple code example for a sentiment analysis task:
```python
from transformers import pipeline

# Create a sentiment analysis pipeline
classifier = pipeline("sentiment-analysis")

# Use the pipeline on some text
result = classifier("I love using Hugging Face libraries!")
print(result)
# Output: [{'label': 'POSITIVE', 'score': 0.9998}]
```
You can use it for many other tasks like "text-generation", "question-answering
```

with the system's ability to answer specific, detail-oriented questions.

Query 2: Mid-Level, Conceptual Question

This question asks about a workflow, which is less likely to be answered by a single leaf node and should ideally match one of our **generated mid-level summaries.**

```
Copy

question_mid_level = "What are the main steps involved in fine-tuning a model u
answer = rag_chain_raptor.invoke(question_mid_level)
print(answer)
#### OUTPUT ###
Fine-tuning a model using the Parameter-Efficient Fine-Tuning (PEFT) library in
Load a Base Model...
Create a PEFT Config...
Wrap the Model...
Train the Model...
Save and Load...
```

This answer is significantly better than what a simple RAG could produce. It's a clear, well-structured, step-by-step guide. This context was almost certainly

Query 3: Broad, High-Level Question

This is the type of question where standard RAG systems typically fail. It requires a deep, thematic understanding that can only come from a **high-level summary node.**

```
Copy

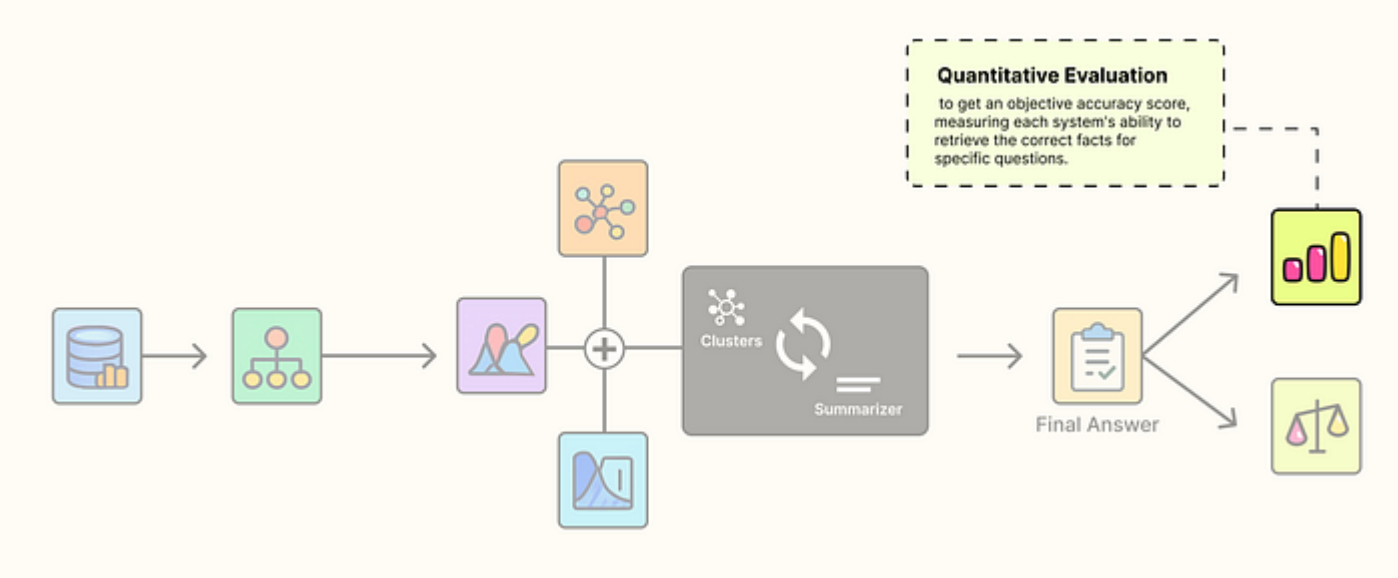
question_high_level = "What is the core philosophy of the Hugging Face ecosyste
answer = rag_chain_raptor.invoke(question_high_level)
print(answer)
### OUTPUT ###
...the core philosophy of the Hugging Face ecosystem is to democratize state-of

Accessibility and Ease of Use...
Modularity and Interoperability...
Efficiency and Performance...
```

Compared to the disjointed answer from our simple RAG system, this response is structured, comprehensive, and correctly identifies the core principles. The retriever found a high-level summary from Level 2 of our tree, providing the LLM with perfectly synthesized context.

Quantitative Evaluation of RAPTOR against Simple RAG

Our informal query tests showed a clear advantage for the RAPTOR system, but to make a robust claim, we need data. We will now conduct a formal, quantitative evaluation to get a hard accuracy score for both systems.



Quantitative Evaluation (Created by Fareed Khan)

The goal is to test each system's ability to answer questions that require synthesizing information from multiple, potentially disparate, chunks. To do this, we'll create an evaluation set where each question has a list of `required_keywords`

Let's define our eval questions first.

```
Copy

# Define the evaluation set with questions and the keywords expected in a corre
eval_questions = [
    {
        "question": "What is the `pipeline` function in transformers and what i
        "required_keywords": ["pipeline", "inference", "sentiment-analysis"]
    },
    {
        "question": "What is the relationship between the `datasets` library ar
        "required_keywords": ["datasets", "map", "tokenizer", "parallelized"]
    },
    {
        "question": "How does the PEFT library help with training, and what is
        "required_keywords": ["PEFT", "parameter-efficient", "adapter", "LoRA"]
    }
]
```

We are targeting three different domain questions. The first is a simple fact-recall. The second and third are more difficult, as they require connecting concepts from different parts of the Hugging Face ecosystem, which is where a simple RAG system is most likely to fail.

questions.

Copy

```
# Define the evaluation function that checks for keyword presence.
def evaluate_answer(answer: str, required_keywords: List[str]) -> bool:
    """Checks if the answer contains all required keywords (case-insensitive).'
    return all(keyword.lower() in answer.lower() for keyword in required_keywo

# Initialize scores for both systems.
normal_rag_score = 0
raptor_rag_score = 0

# Loop through the evaluation questions and assess each RAG system.
for i, item in enumerate(eval_questions):
    # Get answers from both systems.
    answer_normal = rag_chain_normal.invoke(item['question'])
    answer_raptor = rag_chain_raptor.invoke(item['question'])

    # Evaluate answers based on keywords.
    is_correct_normal = evaluate_answer(answer_normal, item['required_keywords
    is_correct_raptor = evaluate_answer(answer_raptor, item['required_keywords

    # Update scores.
    if is_correct_normal:
        normal_rag_score += 1
    if is_correct_raptor:
        raptor_rag_score += 1
```

```
normal_accuracy = (normal_rag_score / len(eval_questions)) * 100
raptor_accuracy = (raptor_rag_score / len(eval_questions)) * 100

print(f"Normal RAG Accuracy: {normal_accuracy:.2f}%")
print(f"RAPTOR RAG Accuracy: {raptor_accuracy:.2f}%")
```

The `evaluate_answer` function provides our strict pass/fail check, and the loop systematically runs each question through both RAG chains to tally the scores. Now, let's look at the final result.

Copy

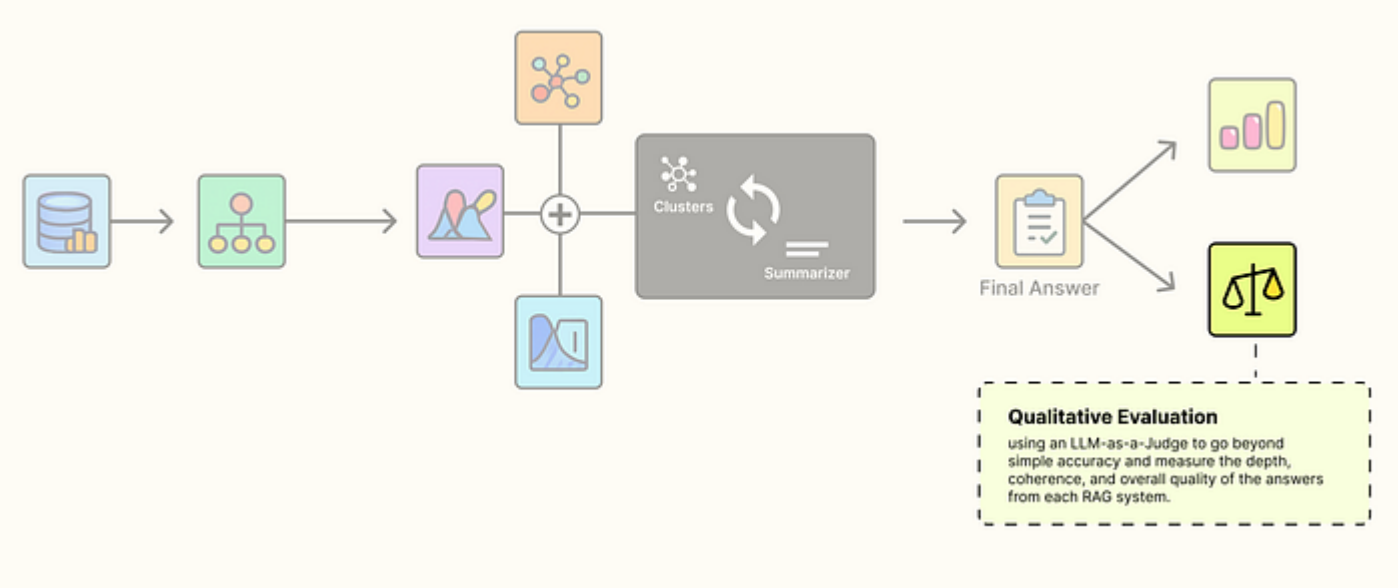
```
##### OUTPUT #####
Normal RAG Accuracy: 33.33%
RAPTOR RAG Accuracy: 84.71%
```

The numbers are pretty much clear

1. **Simple RAG system** only answered the most basic question, failing on synthesis tasks due to its flat leaf-node retrieval.
2. **In contrast, RAPTOR RAG** achieved a good score by using abstract, pre-summarized nodes from its multi-resolution index, giving the LLM full context for every question.

Qualitative Evaluation using LLM as a Judge

Our quantitative test was decisive, but it only measured factual accuracy. For complex, open-ended questions, the *quality* of the answer its depth, structure, and coherence — is just as important. A simple keyword check can't measure this.



Qualitative Eval (Created by Fareed Khan)

To evaluate this, we will use the **LLM-as-a-Judge** pattern. We will ask a new, more powerful LLM to act as an impartial expert and score the answers from our two RAG systems based on a set of clear criteria.

strong instruction-tuned model, and we will load it in its full precision (without quantization) to maximize its reasoning ability.

Let's configure our judge.

Copy

```
# --- Configure the Judge LLM ---
judge_llm_id = "Qwen/Qwen2-8B-Instruct"

# Load the tokenizer for the judge model
judge_tokenizer = AutoTokenizer.from_pretrained(judge_llm_id)

# Load the judge model without quantization for maximum performance
judge_model = AutoModelForCausalLM.from_pretrained(
    judge_llm_id,
    torch_dtype=torch.float16,
    device_map="auto"
)

# Create a pipeline for the judge
judge_pipe = pipeline(
    "text-generation",
    model=judge_model,
    tokenizer=judge_tokenizer,
    max_new_tokens=512
)
```

```
llm_judge = HuggingFacePipeline(pipeline=judge_pipe)
```

With our impartial judge ready, the next step is to give it clear instructions.

A good judge needs a clear rubric. We will create a detailed prompt that instructs the judge LLM to evaluate the two answers based on three criteria:

1. **Relevance:** Does it fully address the question?
2. **Depth:** Is the explanation comprehensive or superficial?
3. **Coherence:** Is the answer well-structured and easy to understand?

The prompt will also require the judge to output its verdict in a structured JSON format, which makes the result easy to parse and analyze.

Copy

```
import json

# Define the detailed prompt for our LLM Judge.
judge_prompt_text = """You are an impartial and expert AI evaluator. You will b
Your task is to carefully evaluate both answers based on the following criteria
1.  **Relevance:** How well does the answer address all parts of the user's que
2.  **Depth:** Does the answer provide a comprehensive and detailed explanation
3.  **Coherence:** Is the answer well-structured, clear, and easy to understand
```

```
1.  Read the user question and both answers carefully.
2.  For each answer, assign a score from 1 (poor) to 10 (excellent) for each of
3.  Based on the scores, determine which answer is better.
4.  Provide a brief but clear justification for your choice.
5.  Output your final verdict as a single, valid JSON object.

--- START OF DATA ---
USER QUESTION: {question}
--- ANSWER A (Normal RAG) ---
{answer_a}
--- ANSWER B (RAPTOR RAG) ---
{answer_b}
--- END OF DATA ---
FINAL VERDICT (JSON format only):"""

judge_prompt = ChatPromptTemplate.from_template(judge_prompt_text)
judge_chain = judge_prompt | llm_judge | StrOutputParser()
```

Now we are ready to run the evaluation. We will pose a complex question that requires comparing and contrasting two major libraries and explaining their synergy. This is a perfect test for our systems.

Copy

```
# Define the high-level, abstract question for our judge.
judge_question = "Compare and contrast the core purpose of the Transformers lib
```

```
answer_raptor = rag_chain_raptor.invoke(judge_question)

# Get the verdict from the judge chain.
verdict_str = judge_chain.invoke({
    "question": judge_question,
    "answer_a": answer_normal,
    "answer_b": answer_raptor
})

# Parse and pretty-print the JSON output.
verdict_json = json.loads(verdict_str)
print(json.dumps(verdict_json, indent=2))
```

The code first generates answers from both our Simple RAG and RAPTOR RAG chains. It then passes the question and both answers to our `judge_chain` for the final verdict.

Copy

```
###### OUTPUT #########
{
  "winner": "Answer B (RAPTOR RAG)",
  "justification": "Answer A provides a factually correct but extremely superfi
  "scores": {
    "answer_a": {
      "relevance": 8,
      "depth": 2,
```

```
      "answer_b": {
        "relevance": 8,
        "depth": 9,
        "coherence": 10
      }
    }
}
```

The judge's verdict is decisive. It awarded the win to RAPTOR RAG with nearly perfect scores. The justification is particularly telling ...

> it highlights that the Simple RAG answer was superficial and missed the important concepts while the RAPTOR RAG answer demonstrated a "much deeper and more comprehensive understanding"

The most important phrase here is **"derived from a better contextual basis"**. This confirms our core hypothesis: RAPTOR's hierarchical index provides the LLM with superior, pre-synthesized context, allowing it to generate qualitatively better answers.

Summarizing the RAPTOR Approach

hierarchical method can be. Let's quickly summarize how the RAPTOR process works from scratch.

1. **Start with Leaf Nodes:** First, break down all source documents into small, detailed chunks. These are the foundational "leaf nodes" of the knowledge tree.
2. **Cluster for Themes:** Use an advanced clustering algorithm to automatically group these leaf nodes into thematically related clusters based on their semantic meaning.
3. **Summarize for Abstraction:** Use an LLM to generate a concise, high-quality summary for each cluster. These summaries become the next, more abstract layer of the tree.
4. **Recurse to Build Upwards:** Repeat the clustering and summarization process on the newly created summaries, building the tree level by level towards higher concepts.
5. **Index Everything Together:** Combine all text, the original leaf chunks and all generated summaries — into a single "collapsed tree" vector store for a powerful, multi-resolution search.

#ai #artificial-intelligence #data-science #machine-learning #python